

Programming Assignment - Algorithm Implementation Project

Introduction

Employers and recruiters can spend significant time reviewing resumes for a position, often checking each application against the same set of job requirements. As the number of applicants increases, manually searching for relevant skills, qualifications, and experience becomes repetitive and time-consuming.

For this project, I developed AhoScan, a resume screening tool that allows employers to define job requirements and search for them across multiple resumes. The main question addressed by the project is: "How can multi-pattern string matching be used to identify employer-defined requirements across a collection of resumes?" AhoScan only identifies occurrences of the configured requirements; it does not rank candidates or make hiring decisions. The project demonstrates how a custom implementation of Aho-Corasick can be integrated into a practical software application.

Algorithm Implementation

AhoScan uses my own implementation of the Aho-Corasick algorithm, which is designed to search for multiple patterns within a text (Sukumaran, 2024). The algorithm first constructs a trie containing all patterns, then adds failure and output links to support efficient matching. During the search, the algorithm follows the text through the trie. When a character cannot be matched, a failure link moves the search to the longest possible suffix that may continue a pattern. When a pattern is found, its identifier is recorded, while the output links are followed to identify additional patterns ending at the same position.

The implementation is contained in `aho_corasick.py`. The `AhoCorasick` class provides three main operations:

- `add_pattern()` inserts patterns into the trie
- `build()` constructs the failure and output links using breadth-first traversal
- `search()` scans the text and returns each match with its pattern ID and start and end positions.

The `Node` dataclass represents trie nodes, while `Match` represents individual search results.

Design Decisions

Language

The algorithm and backend are implemented in Python, with FastAPI providing the HTTP API for jobs, requirements, resumes, and scan requests. Python was chosen so the algorithm could be integrated directly into the backend without requiring an additional language interface. The frontend is built with React and JavaScript, providing the interface for creating jobs, entering requirements, uploading resumes, and viewing scan results.

Data Representation

The trie is represented as a list of nodes, with nodes referencing one another using integer indices. Each `Node` stores a dictionary of character transitions (`next`), a failure link (`fail`), an output link

(out_link), the IDs of patterns ending at that node (outputs), and its depth. The root node is stored at index 0.

The implementation uses output links, where each node points to the nearest node on its failure chain that ends a pattern. This avoids copying the output lists of other nodes. For example, if 'he' and 'she' are both patterns, reaching the end of 'she' can also report 'he' by following the output link.

Pattern IDs are kept separate from application-specific requirement data. In job_store.py, build_matcher() converts each alias into a PatternSpec whose group_id is the index of its requirement. Matcher in matcher.py then adds each spec to one automaton, with the spec's position in the list becoming its pattern ID. This allows the implementation to remain independent of resumes and jobs.

Extensions

The search() method extends the basic matching result by recording the start and end positions of each occurrence rather than only indicating whether a pattern was found.

Application-specific matching rules are handled separately in matcher.py. PatternSpec stores one alias: its text, whether it is a stem, whether it is case-sensitive, and which requirement group it belongs to. Before searching, collapse_whitespace() allows phrases separated by line breaks to be matched. Because collapsing whitespace changes the text positions, it also creates a position map that is used to convert matches back to positions in the original resume text. ascii_lower() provides case-insensitive matching without changing string length, while case-sensitive aliases are checked against the original-case collapsed text after the Aho-Corasick search.

passes_boundary() applies whole-word matching, and extend_to_word_end() handles stem patterns such as manag*. The resolve_overlaps() function merges overlapping matches from the same requirement and flags nested matches from different requirements.

These application-specific rules deliberately sit outside the core algorithm. The overlap resolution is implemented using simple loops over the returned matches rather than a more complex data structure, keeping the implementation straightforward for the scale of the tool. Similarly, lowercasing is deliberately limited to ASCII characters so that character positions remain unchanged.

Code Structure

The core algorithm is contained in aho_corasick.py, with add_pattern(), build(), and search() forming its main interface. job_store.py converts the aliases of each job into PatternSpec objects and builds a Matcher for the job. matcher.py adds those pattern specs to the Aho-Corasick automaton and applies the application-specific matching rules to the raw results.

job_store.py also stores jobs as JSON files and keeps their matchers in memory. Matchers are built when saved jobs are loaded at startup and rebuilt when a job is created or updated. When multiple resumes are scanned, the existing matcher for that job is reused rather than rebuilding the automaton for every resume.

models.py defines the structure of jobs, requirements, aliases, resumes, and scan requests. scan_service.py converts matcher results into the requirement-level results displayed by AhoScan, including match locations and snippets. app.py exposes these functions through the FastAPI endpoints, while the React frontend communicates with the backend through HTTP requests.

This structure keeps the Aho-Corasick implementation independent from the resume-screening application while allowing the surrounding layers to handle job requirements, matching rules, and presentation.

The Tool

What it does

AhoScan is a resume screening tool that searches a collection of resumes against employer defined job requirements. The user first creates a job by entering its title and requirements. Each requirement can have multiple aliases, allowing different ways of expressing the same requirement to be searched. Requirements can also be marked as required or nice-to-have.

The user then uploads multiple resumes for the selected job. AhoScan searches each resume against the requirements and displays whether each requirement was found, how many accepted occurrences were found, and where they occurred in the resume. Matching text can be displayed with surrounding context so that the user can inspect where the requirement appeared.

The tool is deliberately descriptive rather than evaluative. It identifies occurrences of configured requirements but does not assign candidate scores, rank applicants, or make hiring decisions.

How the Algorithm Sits Inside the Tool

The implementation forms the search component of AhoScan. When a job is configured, its aliases are treated as the patterns to search for. A single matcher is built for each job and reused to search each uploaded resume against the same set of requirements

The raw matches returned by the algorithm are processed by the rest of the application before being shown to the user. Matching rules determine which occurrences are accepted, while the scan service groups accepted matches according to their requirements and generates the information displayed in the results. This creates a clear separation between the algorithm and the application: Aho-Corasick identifies pattern occurrences, while the rest of the app turns those occurrences into resume screening results.

Interface Decisions

The interface is organised around the process of defining a job and then checking multiple resumes against it. Requirements are represented using a name, a required or preferred status, and one or more aliases rather than exposing the underlying pattern-matching implementation to the user.

Aliases allow different forms of a requirement to be searched. For example, a Search Engine Optimisation requirement could use SEO and search engine optimi* as aliases. The stem syntax allows related word forms to be matched without requiring the user to enter every possible variation separately.

The results are grouped by requirement rather than displayed as a raw list of string matches. Each requirement can show its state, number of matches, matching text, and surrounding context. This allows the user to inspect the evidence behind a result rather than simply receiving a yes/no response.

Required and preferred requirements are also separated in the summary. Multiple resumes can be scanned against the same job configuration, supporting the main use case of checking a collection of applications against one set of requirements.

Worked Example

Below is an example of how a user would use AhoScan to screen resumes against the requirements of a job. The example demonstrates the main workflow from creating a job to reviewing the results produced for each resume.

1 - Create a Job

The process begins when the user clicks the 'Get Started' button on the landing page, which redirects them to the job creation page. The user enters a job title and adds the requirements they want to search for. Each requirement can contain one or more aliases, allowing different ways of expressing the same requirement to be grouped together.

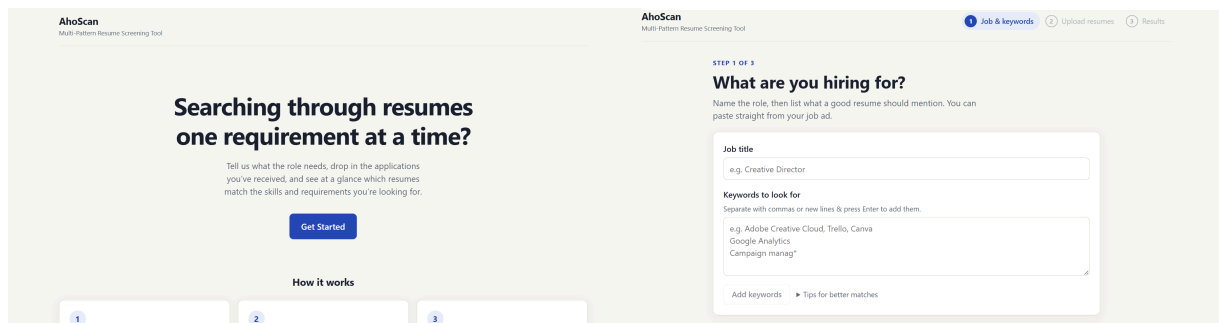


Figure 1: Landing page for the application

Figure 2: Job creation interface showing the job title and configured requirements

Job title

Software Engineer

Keywords to look for

Separate with commas or new lines & press Enter to add them.

Node.js
Github

Add keywords ▶ Tips for better matches

3 required · 2 nice to have. Click a keyword to switch between the two.

REQUIRED Javascript × REQUIRED Python × REQUIRED APIs × NICE TO HAVE teamwork ×

NICE TO HAVE Visual Studio Code ×

Continue to upload

Figure 3: User enters the job title and requirements and selects whether each requirement is required or preferred

2 - Upload Resumes

After selecting 'Continue to upload', the user uploads the resumes they want to compare against the same set of requirements. AhoScan extracts the text from each resume and sends the resume content to the backend for scanning.

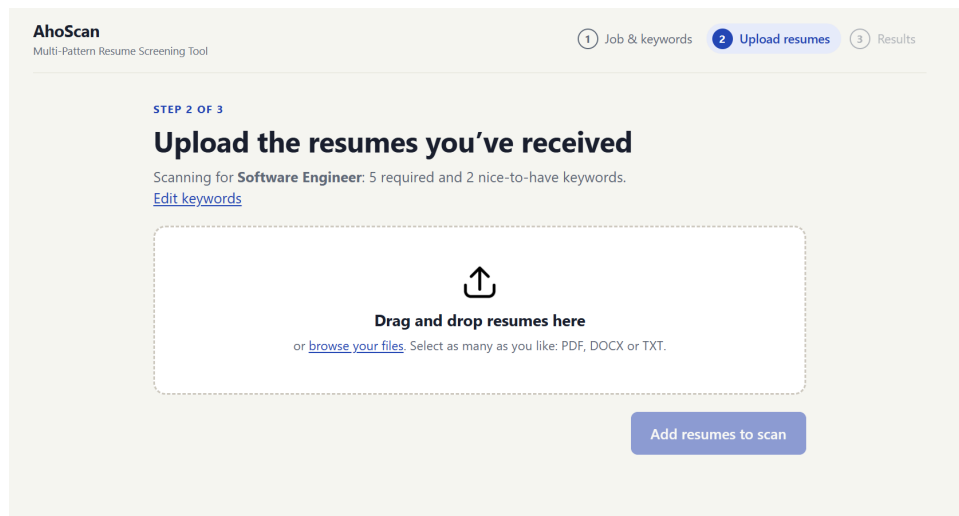


Figure 4: Resume upload page

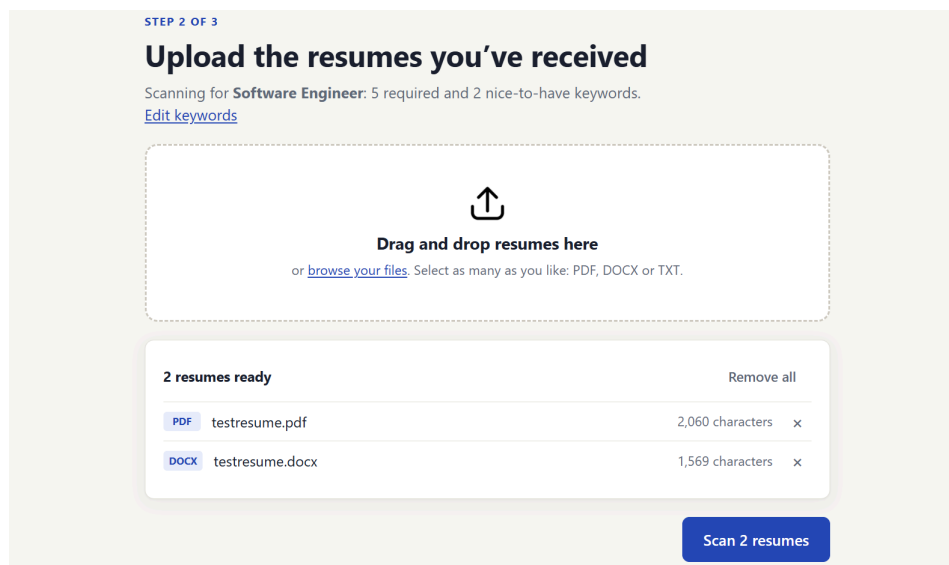


Figure 5: Resume upload page with two resumes selected

3 - Scan Resumes

The user starts the scan. AhoScan uses the Matcher associated with the selected job to search each resume. The Matcher uses the Aho-Corasick automaton containing all of the job's aliases, allowing the same set of patterns to be searched across every resume.

4 - Review Results

The results are grouped by resume and requirement. Each requirement is shown as found, flagged, or missing, allowing the user to quickly see which configured requirements were identified in each resume.

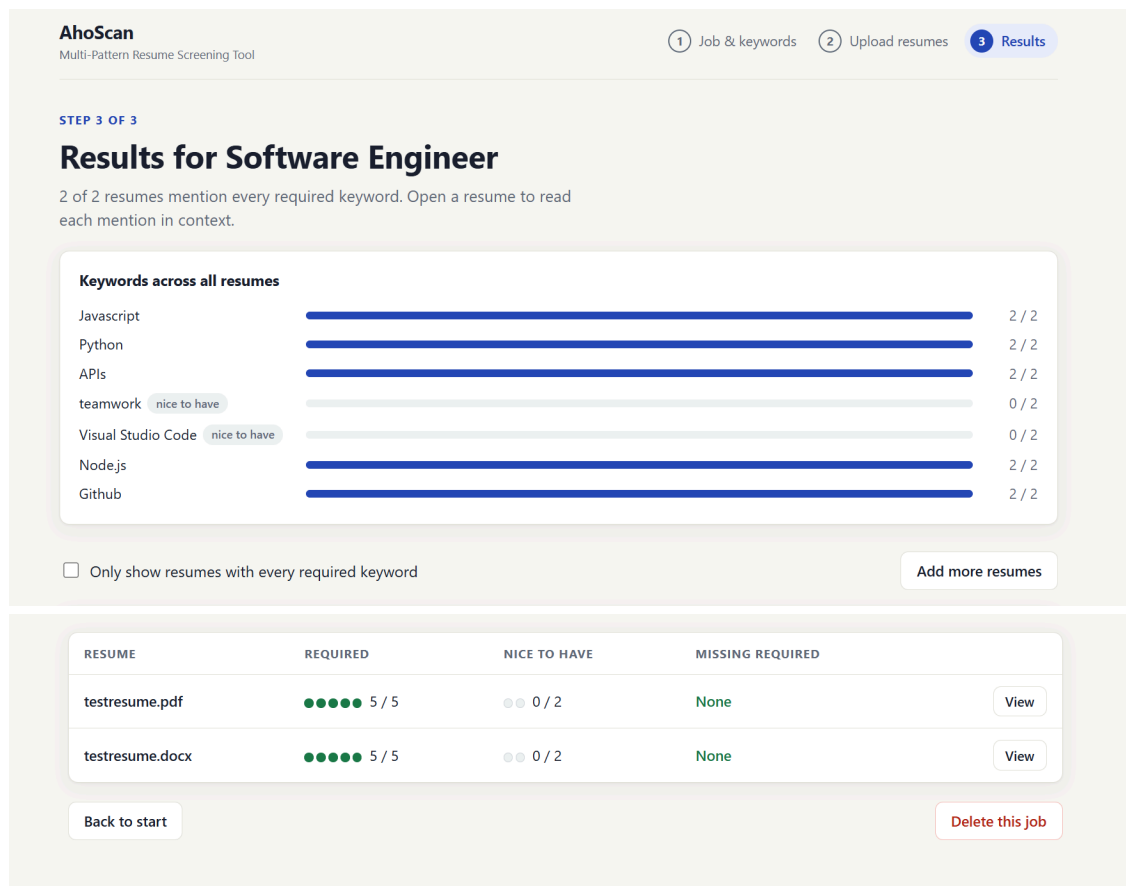


Figure 6: shows the results summary for one scan

5 - Inspect matched text

The user can inspect each resume by selecting 'View' to see snippets associated with matched requirements. These snippets provide surrounding resume text, giving context for the keyword match rather than only displaying that the keyword was found.

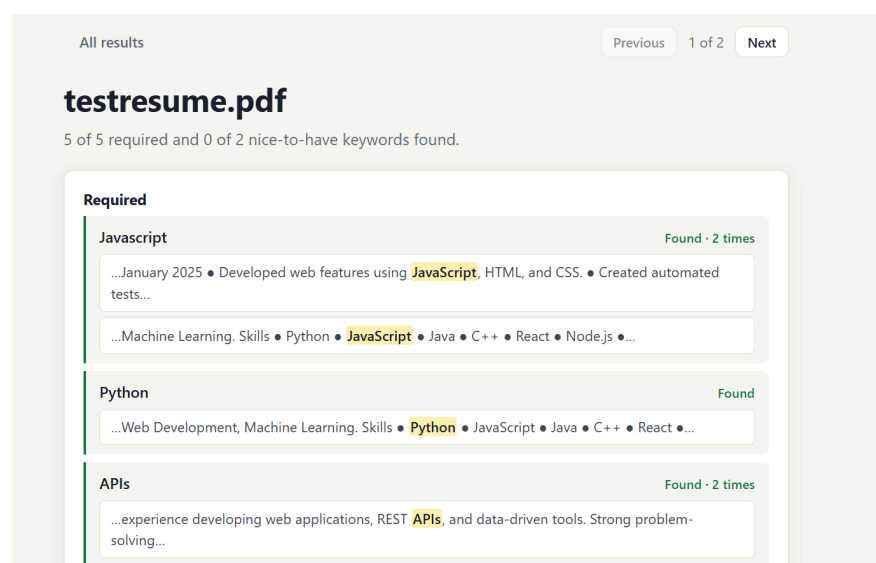


Figure 7: expanded results with its matched text and surrounding snippet

This example demonstrates how the tool works from start to finish and how the Aho-Corasick implementation forms the matching component of a larger application. The algorithm identifies occurrences of the configured patterns, while the surrounding matching and application layers interpret those occurrences as job requirements and present the results to the user.

What I Learned

I started this project with a good theoretical understanding of the algorithm, so the main challenge was not understanding the algorithm itself but turning it into a working implementation within a real application. I found that the implementation introduced more problems than I initially expected, particularly once the algorithm had to work with extracted resume text rather than controlled test strings.

One example was text normalisation. I initially expected lowercasing to be a simple preprocessing step, but discovered that Python's `str.lower()` can change the length of some Unicode characters, which can make match positions incorrect. To combat this I used an ASCII-only conversion that preserves string length. PDF text extraction created another problem with phrases such as `'social\nmedia'` could be split across lines and fail to match `'social media'`. I addressed this by collapsing whitespace before searching, but this changed the text positions, so I also needed a position map to convert matches back to the original resume.

These problems gave me a better understanding of the practical application of the algorithm. It is effective when the patterns that need to be found are explicitly defined, but it does not provide semantic understanding. For example, it can't determine that `'customer service'` and `'client support'` may describe similar skills, or handle a misspelling without additional logic. Features such as aliases, stems, boundaries and text normalisation were therefore needed around the core algorithm, but they still don't account for all cases.

This also made me question whether Aho-Corasick is the most suitable approach for modern resume screening. I deliberately chose to build the tool this way because I wanted to understand what the core implementation of keyword-based resume screening looks like before the use of modern AI and language models. The project showed me that the algorithm can provide an efficient foundation for explicit keyword matching, but that many of the problems involved in interpreting resumes lie outside the algorithm itself.

AI Use

As I had studied the algorithm before starting the implementation, I used AI less for learning the fundamental algorithm and more for supporting the development of the surrounding application. I used Claude Code for some frontend code and styling, application structure, test data, and testing ideas. I made an effort to write the backend code myself, as I often found that AI-generated implementations either did things differently from how I wanted to approach the problem or included code that I did not fully understand. When I did use generated code, I checked it by reading the code, running tests, and using the application rather than taking it entirely on trust.

Some examples where the AI was wrong was when adding a `'useEffect'` containing `'window.scrollTo()'`. This caused the page to go blank when I ran the application. I identified the problem by testing and navigating through the interface, and then changed the implementation.

It also generated an API test that expected two occurrences of `"Google Analytics (GA4)"` to count as one match. The two occurrences did not overlap, so the implementation correctly returned two

matches. Running the test showed that the expected result was wrong, so I corrected the test rather than changing the matching code.

Another example was how resume files were read. Claude initially suggested extracting PDF and Word document text in the browser using pdf.js and mammoth. This worked, but the PDF setup required an unusual worker configuration and made the frontend more complicated than I wanted. I decided to move the file processing to the Python backend using pypdf and python-docx instead. This simplified the architecture and removed the two document-processing dependencies from the frontend. This was an example where the AI's proposed solution was workable, but I preferred a different design because it was easier for me to understand and explain.

Overall, AI was useful for testing and frontend refinement, but its mistakes meant that I needed a good understanding of what the code was supposed to do and how to verify that it was working correctly. This was particularly important when using AI-generated tests, as a test can itself contain an incorrect assumption even when the implementation is correct.

References

Sukumaran, B. (2024, May 15). The essence of Aho-Corasick algorithm. Medium.
<https://medium.com/@balajisukumaran96/the-essence-of-aho-corasick-algorithm-4056dd742842>